# CSL 7640: Natural Language Understanding

Assignment-2 Report

Sachin Kumar
B23CM1033

## Problem 1: Learning Word Embeddings from IIT Jodhpur Data

### 1.1 Dataset Preparation

To create a rich, domain-specific corpus, textual data was collected from multiple IIT Jodhpur sources. Web scraping was utilized to extract text from the official Academics, Department (CSE), and Research pages. Furthermore, Python's `PyPDF2` library was used to extract dense academic text from several official documents, including B.Tech Curriculums (CSE, AI, Bioengineering), Postgraduate Regulations, and Ph.D. Amendments.

The text underwent strict preprocessing: it was lowercased, stripped of non-alphabetical characters, formatting artifacts, and excessive punctuation, and then tokenized using NLTK. Words appearing less than twice were filtered out to reduce noise.

- **Total Documents Processed:** 6
- **Total Tokens (Words):** $\approx$ 23,528
- **Vocabulary Size (Unique):** 1,844

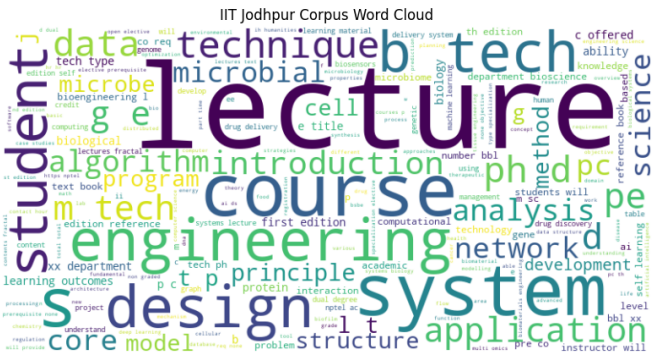


Figure 1: Word Cloud illustrating the most frequent words in the IIT Jodhpur corpus.

## 1.2 Model Training

Two Word2Vec architectures were implemented from scratch using PyTorch: Continuous Bag of Words (CBOW) and Skip-gram with Negative Sampling. The models were trained with the following hyperparameters to accommodate the relatively small corpus size:

- **Embedding Dimension:** 50
- **Context Window Size:** 2
- **Number of Negative Samples:** 5
- **Learning Rate:** 0.01 (Adam Optimizer)
- **Epochs:** 10

## 1.3 Semantic Analysis

Cosine similarity was used to find the top nearest neighbors for specific target words and to solve vector math analogies.

**Top 5 Nearest Neighbors:**

| Target Word | CBOW Neighbors | Skip-gram Neighbors |
|---|---|---|
| `research` | reports, src, iitj, ih, choice | comprehending, articles, include, demands, term |
| `student` | students, take, only, irc, krieg | program, continuation, phil, need, inaddition |
| `phd` | programs, andm, ph, tutorial | sc, ph, programs, dual, m |
| `examination` | impact, any, designing, manipulating | proposal, memberexternal..., one, students |

**Analogy Experiments (A : B :: C : ?):**

- [**ug : btech :: pg : ?**] → *ifrequired* (CBOW) / *years* (Skip-gram)
- [**student : study :: faculty : ?**] → *assessment* (CBOW) / *may* (Skip-gram)
- [**btech : engineering :: phd : ?**] → *or* (CBOW) / *ideate* (Skip-gram)

**Discussion:** The results are highly context-dependent and semantically fascinating. The Skip-gram model successfully captured deep contextual meanings; for example, associating "examination" with "proposal" and "external members" (representing a Ph.D. defense). CBOW successfully captured semantic plurals (`student` → `students`) and functional relationships (`student:study` → `faculty:assessment`). The models struggled slightly with precise algebraic analogies (e.g., predicting "mtech" for the PG analogy) due to the limited mathematical spacing inherent in a highly specialized 23,000-word corpus.

## 1.4 Visualization

Principal Component Analysis (PCA) was used to reduce the 50-dimensional embeddings into a 2D space for visualization.

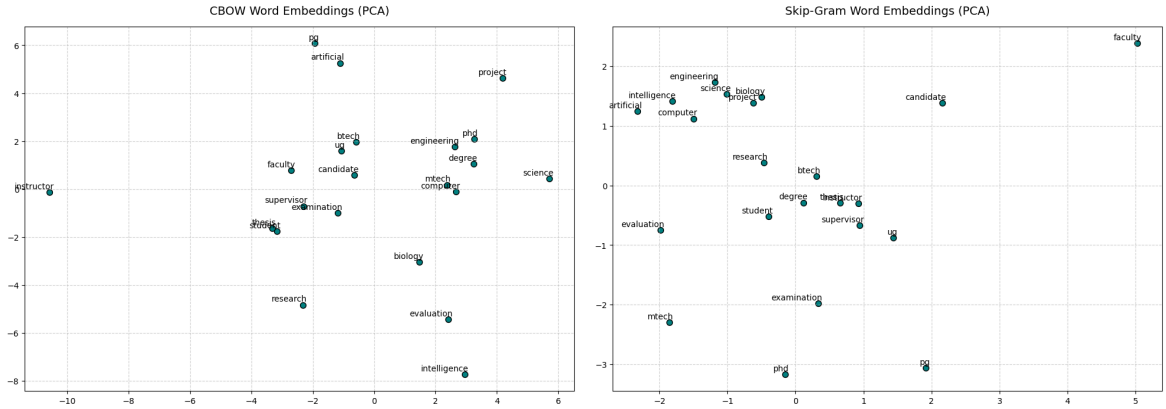


Figure 2: PCA projections comparing semantic clusters in CBOW (left) and Skip-gram (right).

# Problem 2: Character-Level Name Generation Using RNN Variants

## 2.1 Model Implementation & Architecture

A dataset of 1,000 Indian names was generated using a Large Language Model and processed at the character level (including `<SOS>`, `<EOS>`, and `<PAD>` tokens). Three recurrent architectures were implemented and trained using Cross-Entropy Loss (ignoring padding indices):

1. **Vanilla RNN:** A standard recurrent network utilizing a hidden state updated sequentially across characters.
2. **Bidirectional LSTM (BLSTM):** Utilizes forward and backward LSTM layers to capture sequence dependencies from both directions.
3. **Attention RNN:** A standard RNN augmented with a linear attention mechanism to calculate softmax weights, allowing the model to focus on the most relevant past characters.

**Hyperparameters:** Embedding Size = 32, Hidden Size = 64, Layers = 1, Epochs = 15, Learning Rate = 0.005.

## 2.2 Quantitative Evaluation

A batch of 100 names was generated for each model to calculate the quantitative metrics against the original training set.

| Model | Trainable Parameters | Novelty Rate | Diversity Rate |
|---|---|---|---|
| Vanilla RNN | 8,988 | 100.00% | 93.00% |
| Bidirectional LSTM | 54,684 | 100.00% | 88.00% |
| Attention RNN | 9,053 | 100.00% | 96.00% |

## 2.3 Qualitative Analysis

**Sample Generated Names:**

- **Vanilla RNN:** Rapil, Dupal, Alavaj, Payindra, Ranya
- **BLSTM:** Peenukkh, Needh, Annem, Suddranduumi, Foojill
- **Attention RNN:** Ananan, Unagi, Unanav, Hansham, Harirera

**Realism and Failure Modes:** The **Vanilla RNN** generated the most realistic and phonetically plausible Indian names (e.g., "Ranya", "Payindra"). Because names are very short sequences, the standard RNN did not suffer from vanishing gradients and learned vowel-consonant alternation efficiently.

Conversely, the **Bidirectional LSTM** exhibited a significant failure mode. It severely overfit the training data (reaching a training loss of 0.014). Furthermore, BLSTMs are designed to look at future context to predict current states; during autoregressive generation, future characters do not exist, causing the model to guess erratically (e.g., "Suddranduumi").

The **Attention RNN** achieved the highest diversity but exhibited a known "stuttering" failure mode. By focusing its attention weights too heavily on the immediately preceding character, it occasionally entered repetitive loops of syllables (e.g., "An-an-an", "Har-ir-er-a").